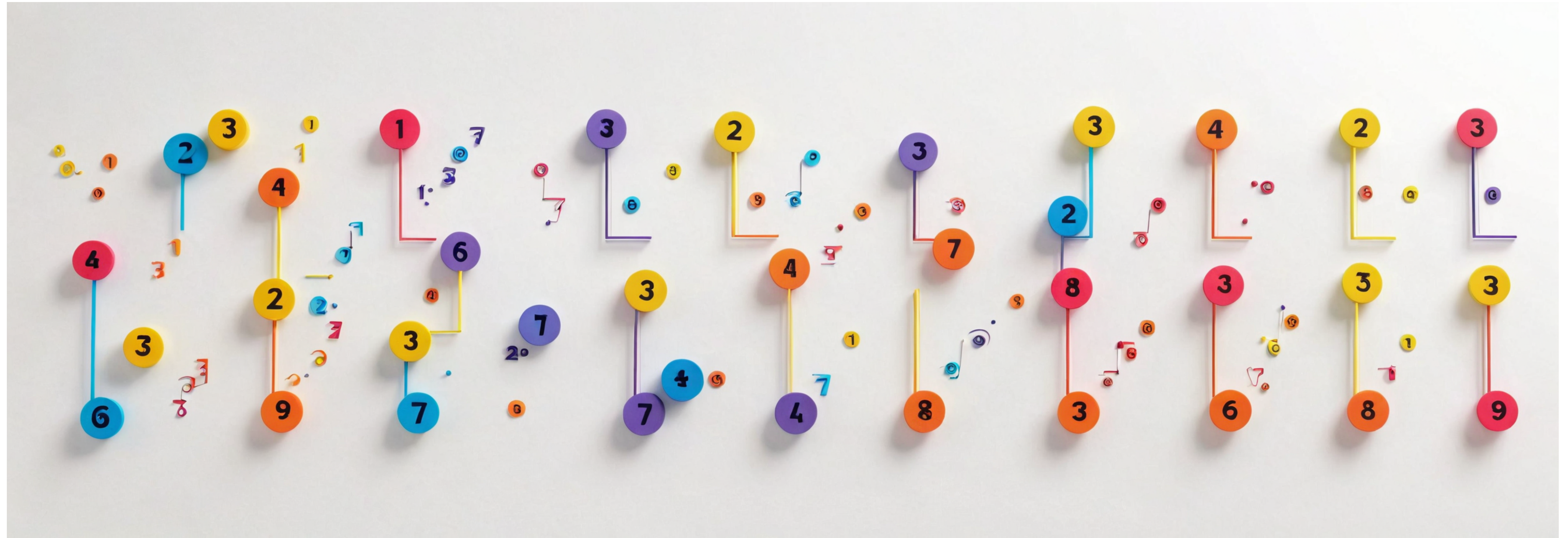




Audio Computing Lab

Speech and Audio Coding

Paris Smaragdis
paris@illinois.edu
paris.cs.illinois.edu

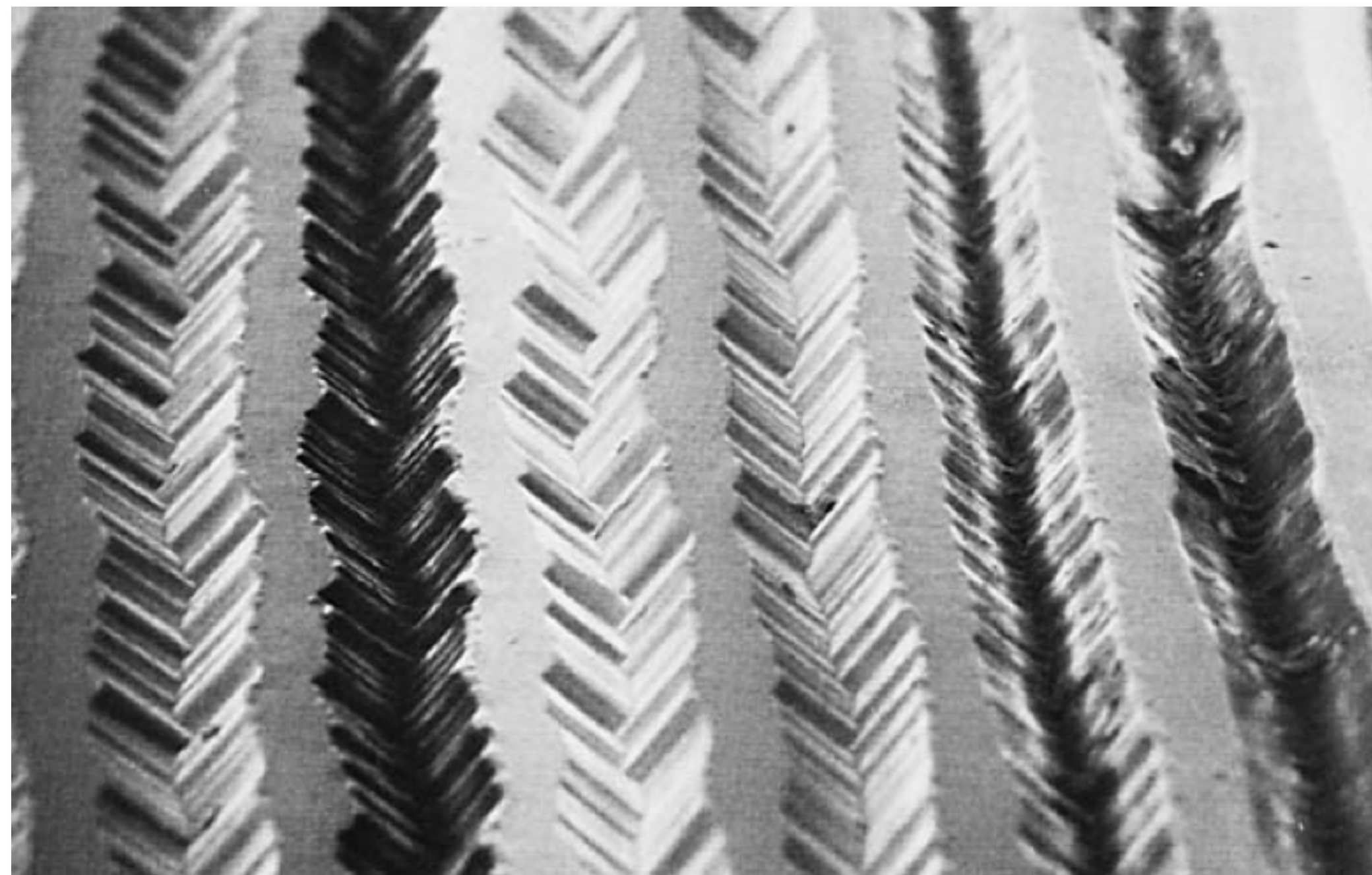
Overview

- Audio storage, encoding, and bandwidth
 - Simple coding tricks
- Speech compression
 - Linear prediction
- General audio and music compression
 - Perceptual coding (e.g. MP3s)

Storing audio

- Earliest methods were analog
 - E.g., wax, vinyl
- We literally “scratched” the waveforms

1897 wax cylinder recording

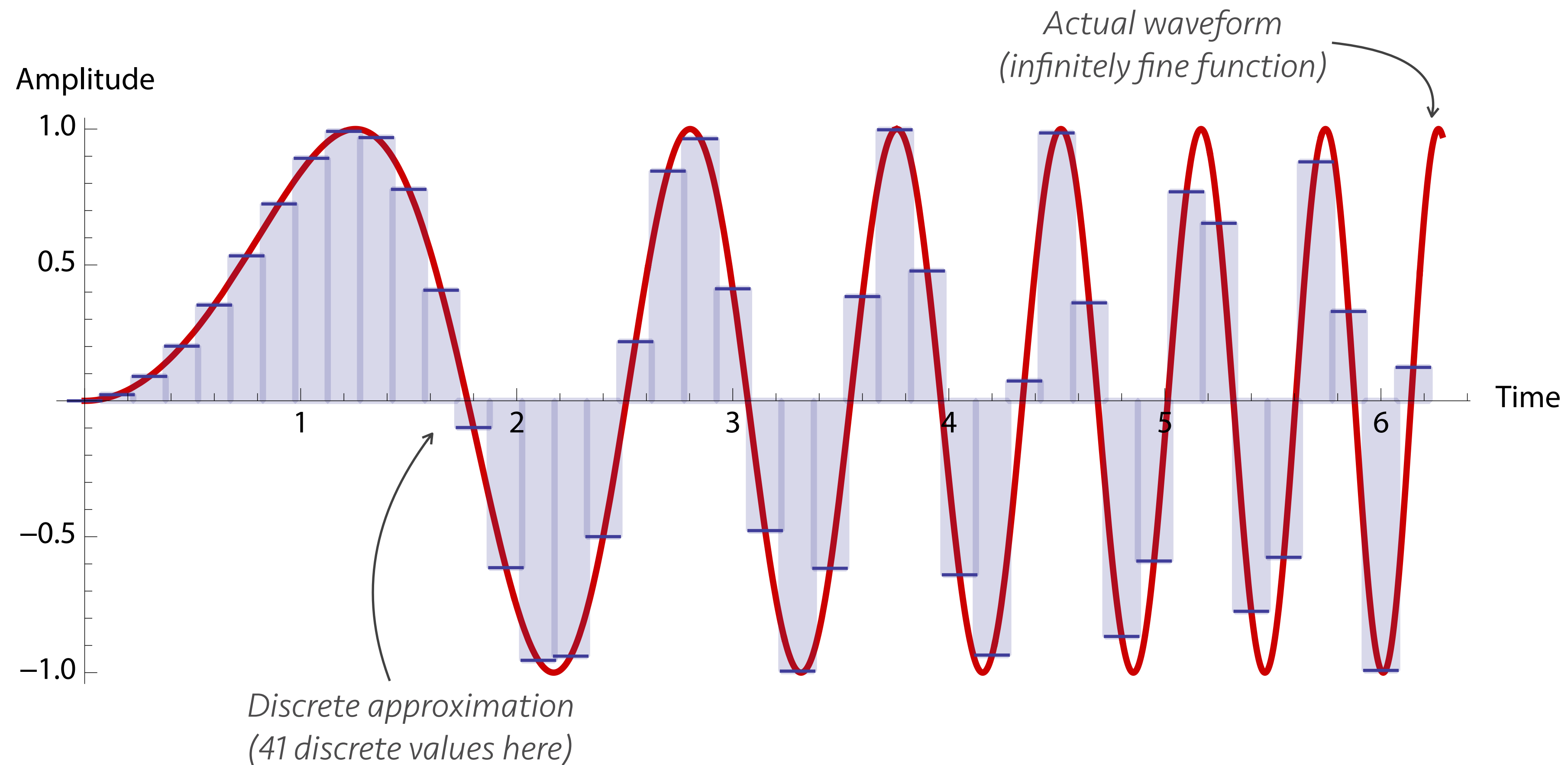


Digital media

- Analog media “fades” with additional playback
 - Tape demagnetizes, vinyl grooves smooth, etc.
 - Digital media was the cure to that
- Digital Tape, Compact Disks, memory storage
 - Stores music as a sequence of discrete numbers
 - Storage is still analog, representation is digital
 - Universally use the PCM format
 - Pulse Code Modulation
 - Same encoding over all consumer devices

PCM encoding example

- Waveform is quantized in time and amplitude

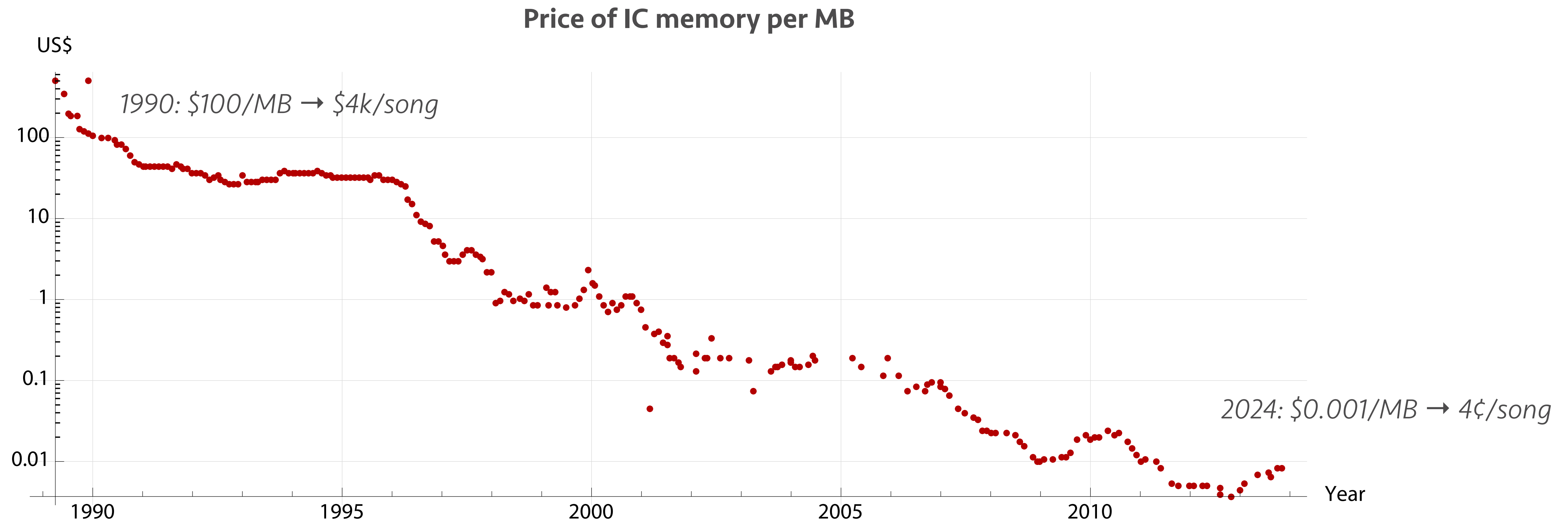


Storage requirements

- Music is usually sampled 44,100 times a second
 - ×2 for stereo recordings
 - Using 16-bit precision (2 bytes per value)
- The average pop song is 220sec long
 - This is about 38 Mbytes
- Easy to store an album in a CD (700MB)
 - But expensive for memory

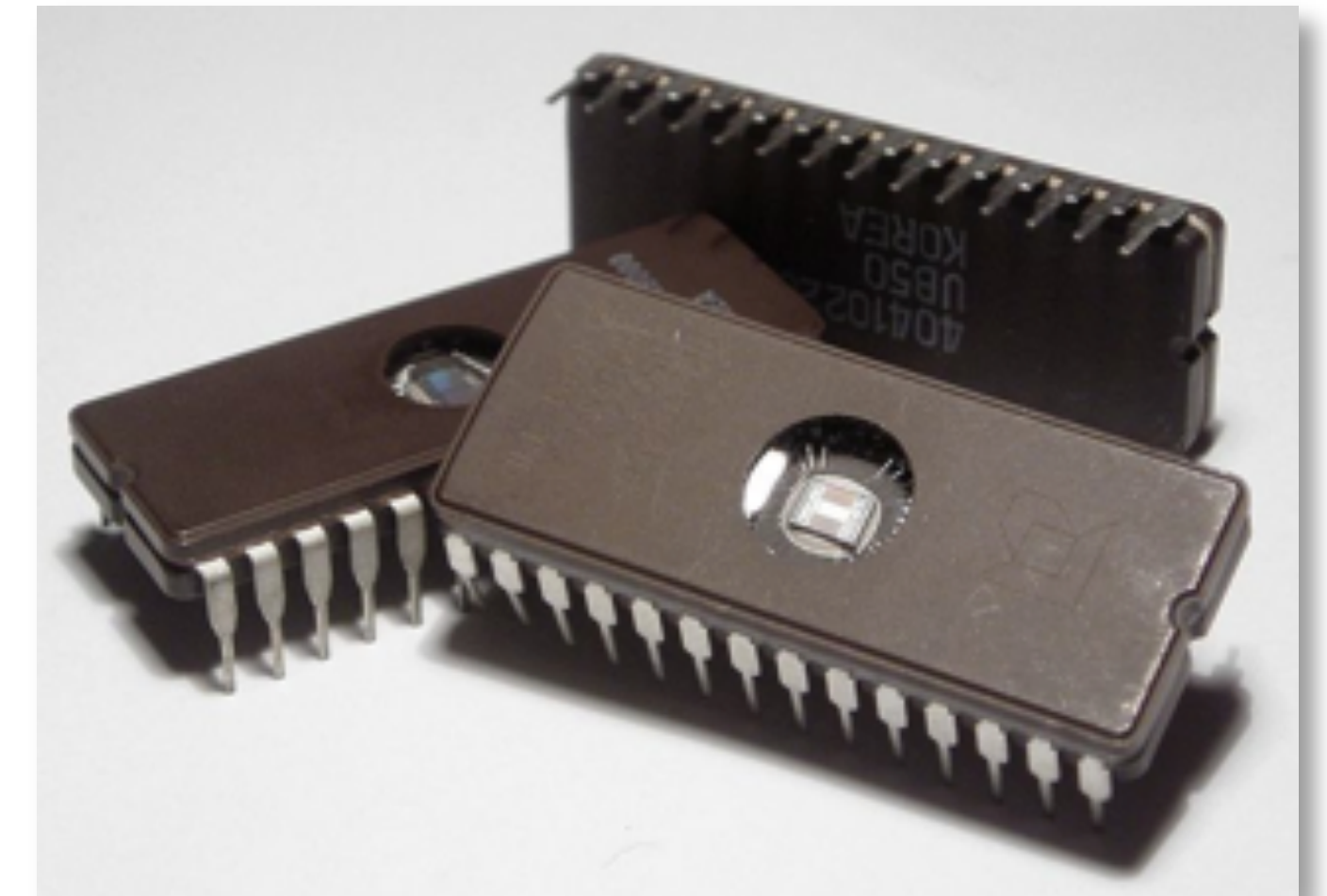
Integrated Circuit (IC) Memory

- Before the 1990s IC memory was expensive and not a viable storage format for media like music 1957: \$400M/MB → \$16B/song
- Now it is commonplace (iPods, cell phones, etc.)



IC memory

- Information is stored in transistors
 - Advantage: No moving parts
 - Less likely to break from parts fatigue
 - No wear and tear
- Memory varieties
 - Read-Only Memory (ROM)
 - Once written cannot be changed (like a vinyl record)
 - Random Access Memory (RAM)
 - Rewritable at will (like a tape)
 - Most common type for audio



Compressing audio

- 38MB per song is still a lot of memory
 - To reduce that we use compression
- Two main varieties
 - Lossless compression
 - Lossy compression (usually perceptual coding)
- Can shrink memory requirements down to 10-20%
 - Made music storage on IC memory viable

What about transmission?

- Same problem
 - The less data we send, the better
 - Save battery, bandwidth, costs, etc.
- Compression and coding
 - Minimize data volume
 - Maximize robustness to network glitches
 - Can we recover from a temporary data loss?

The simple tricks

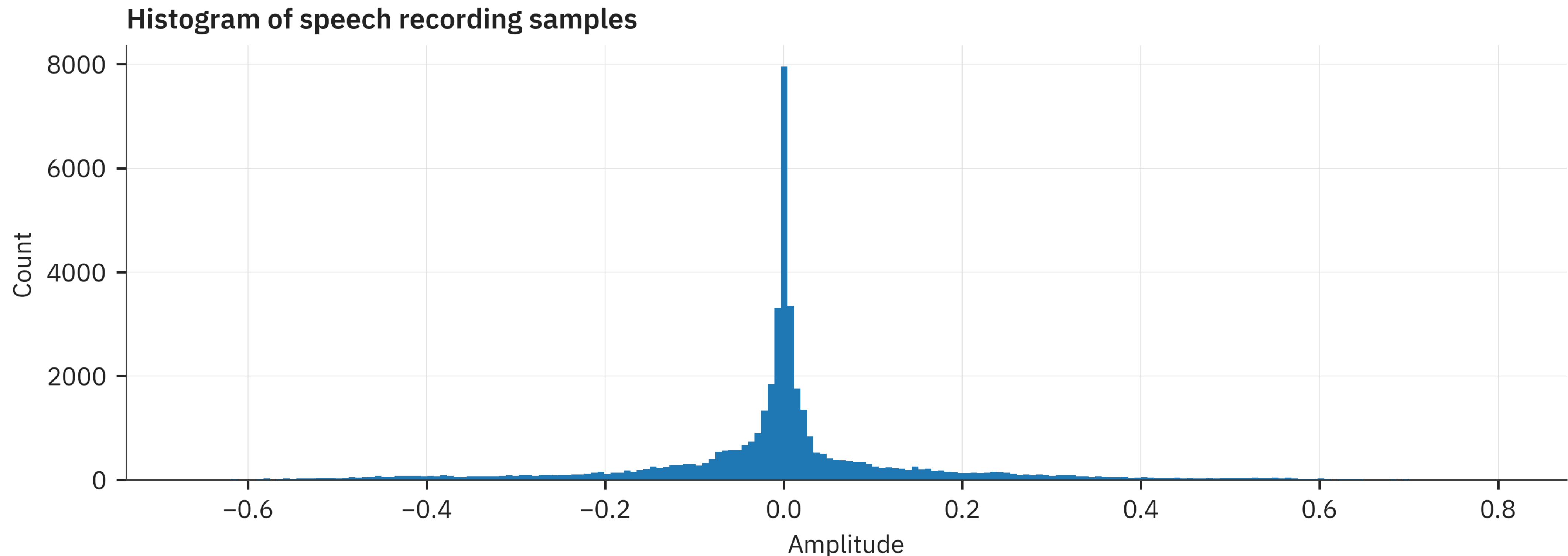
- Analog audio
 - Change the size/speed of the media
 - E.g. disk speed, tape width, etc.
- Digital audio
 - Change bit depth
 - We know how that sounds!
 - Change sample rate
 - We know what this does!

Reducing bits in a smarter way

- By reducing bits we lose dynamic range
 - Ok for speech, really bad for, e.g., classical music
- *Com**panding* approaches
 - Range-*comp*ress the input to encode it better
 - Ex*pan*d while decoding
 - Maintains dynamic range width
 - Misses out on resolution

Key observation

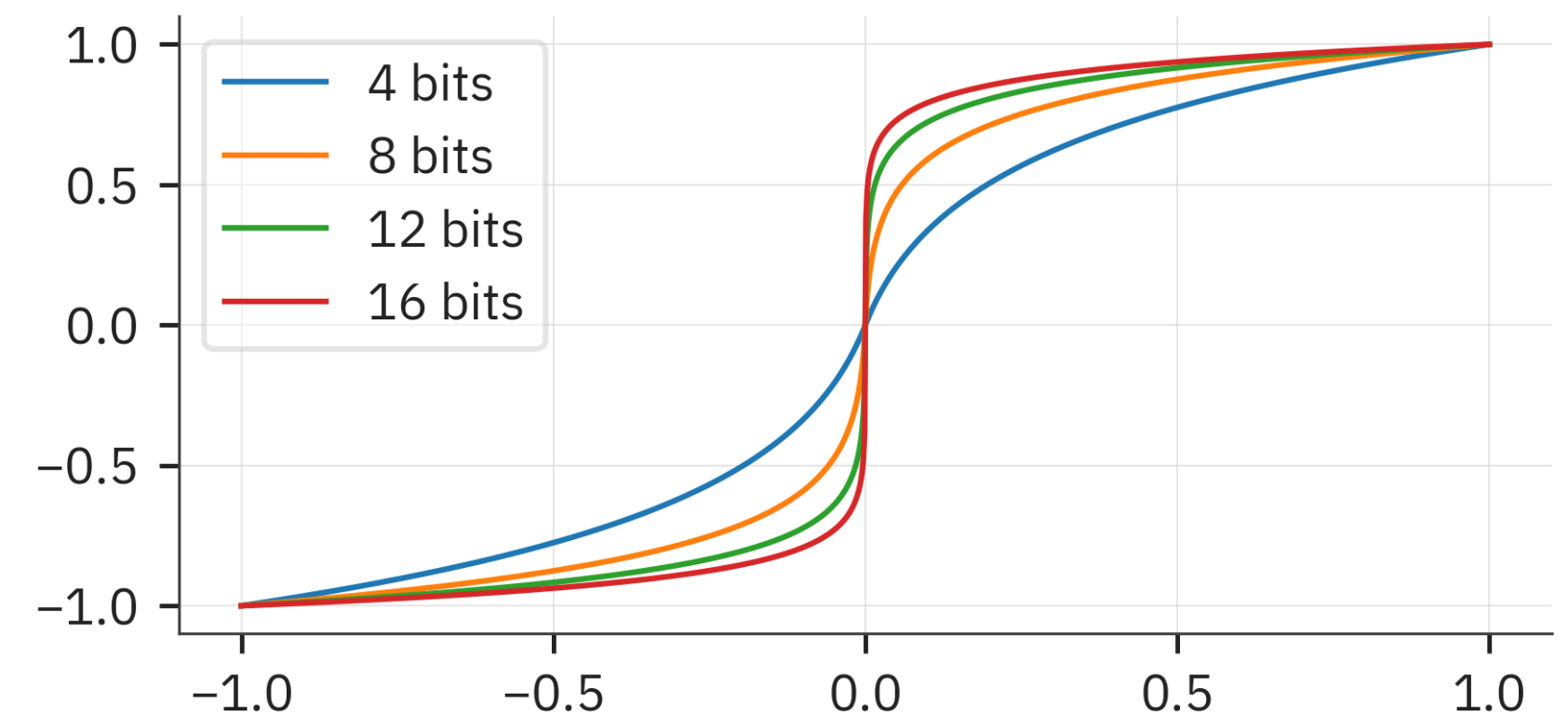
- Audio is *heavy-tail* distributed
 - We rarely see high-amplitude values
 - So why waste high bit accuracy for such rare cases?



Redistribute the values

- Pass data through a sigmoid
 - μ is the number of discrete values

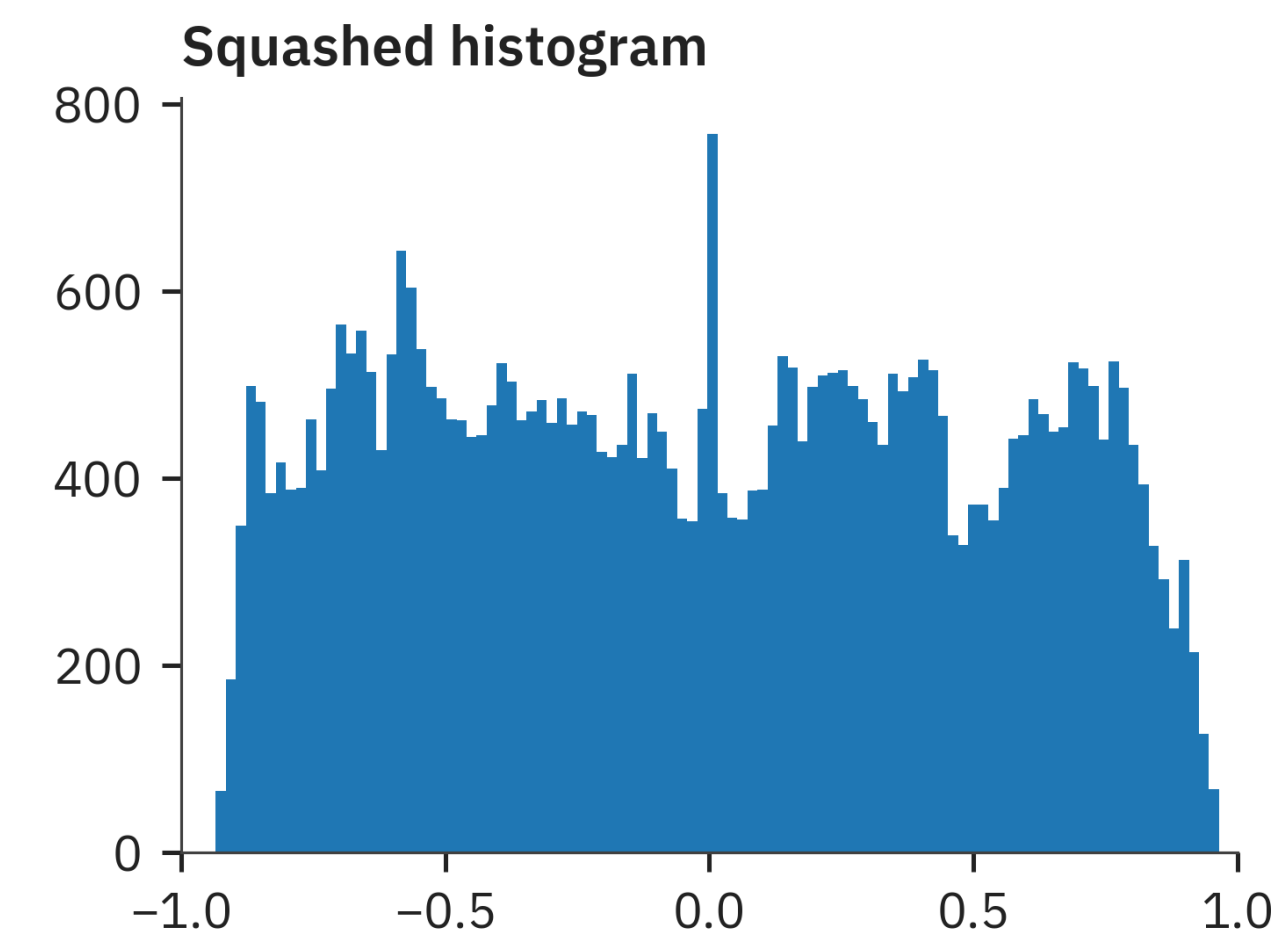
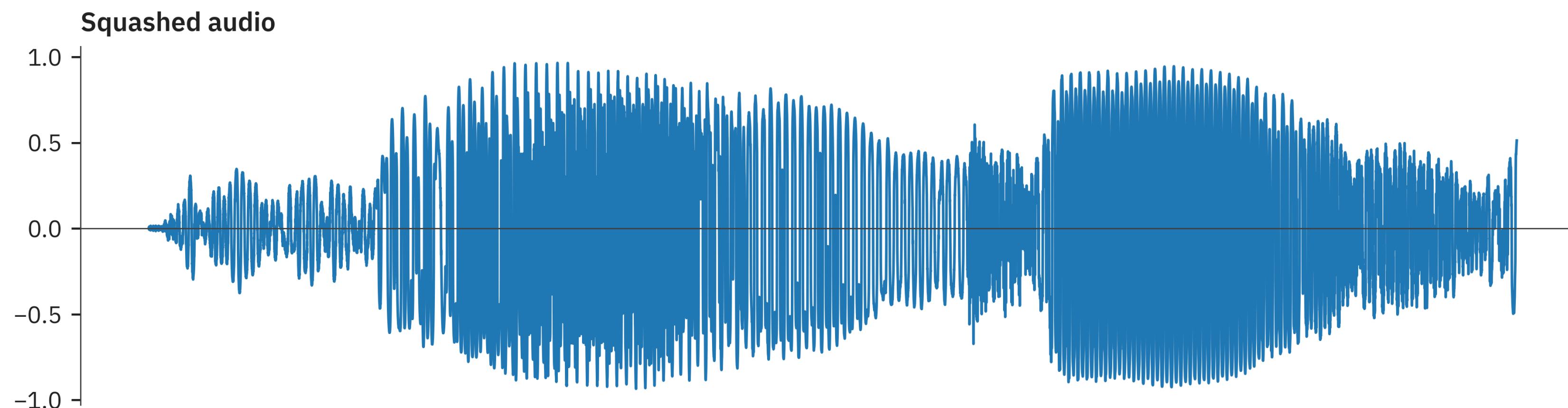
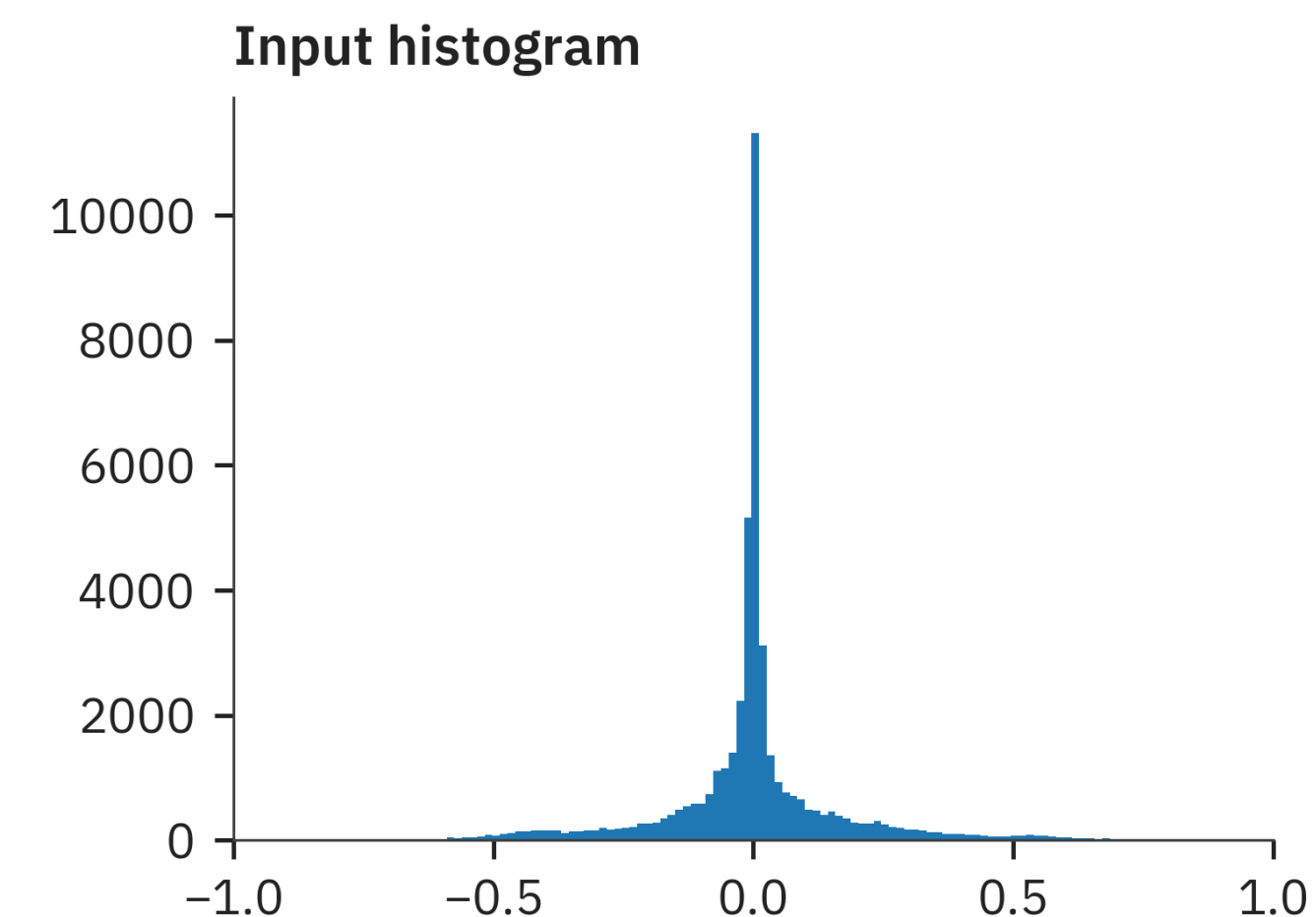
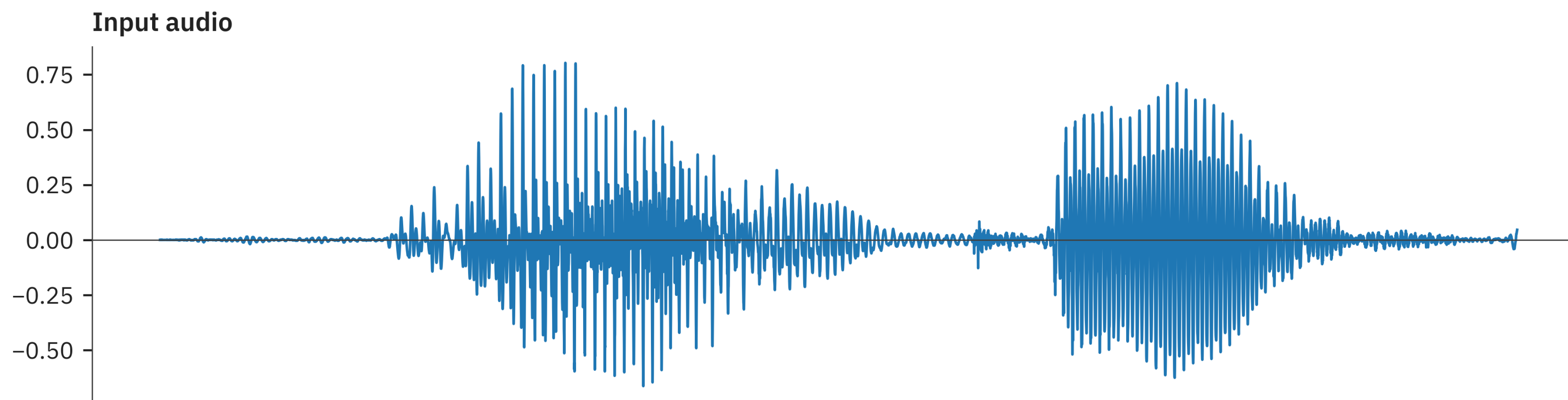
$$y = \frac{\text{sgn}(x) \log(1 + \mu |x|)}{\log(1 + \mu)}, -1 \leq x \leq 1$$



- Effectively squashes the extreme waveform values and minimizes the outliers

Effect on waveform

- Fewer outliers $\rightarrow$ no need for high bit precision

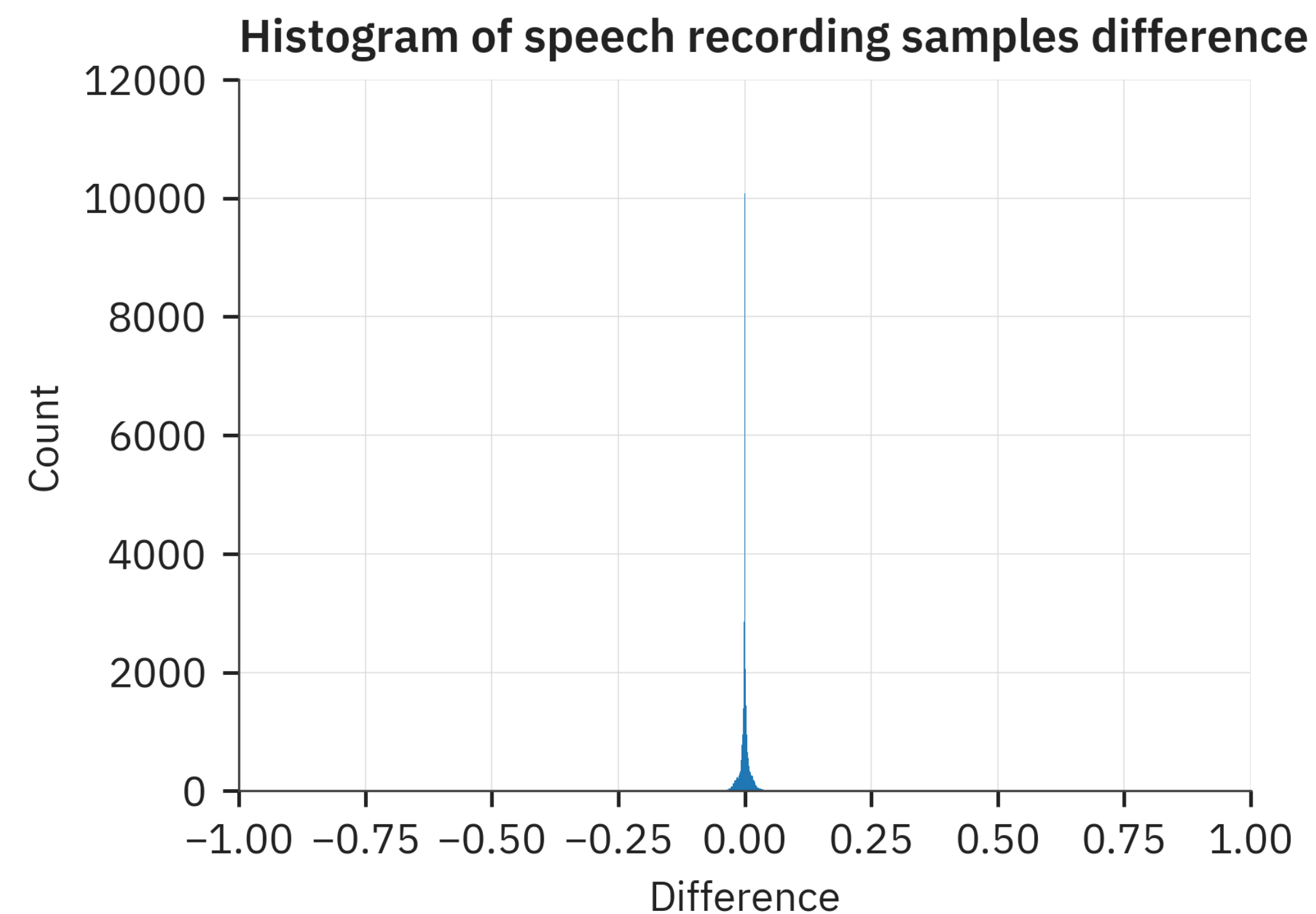
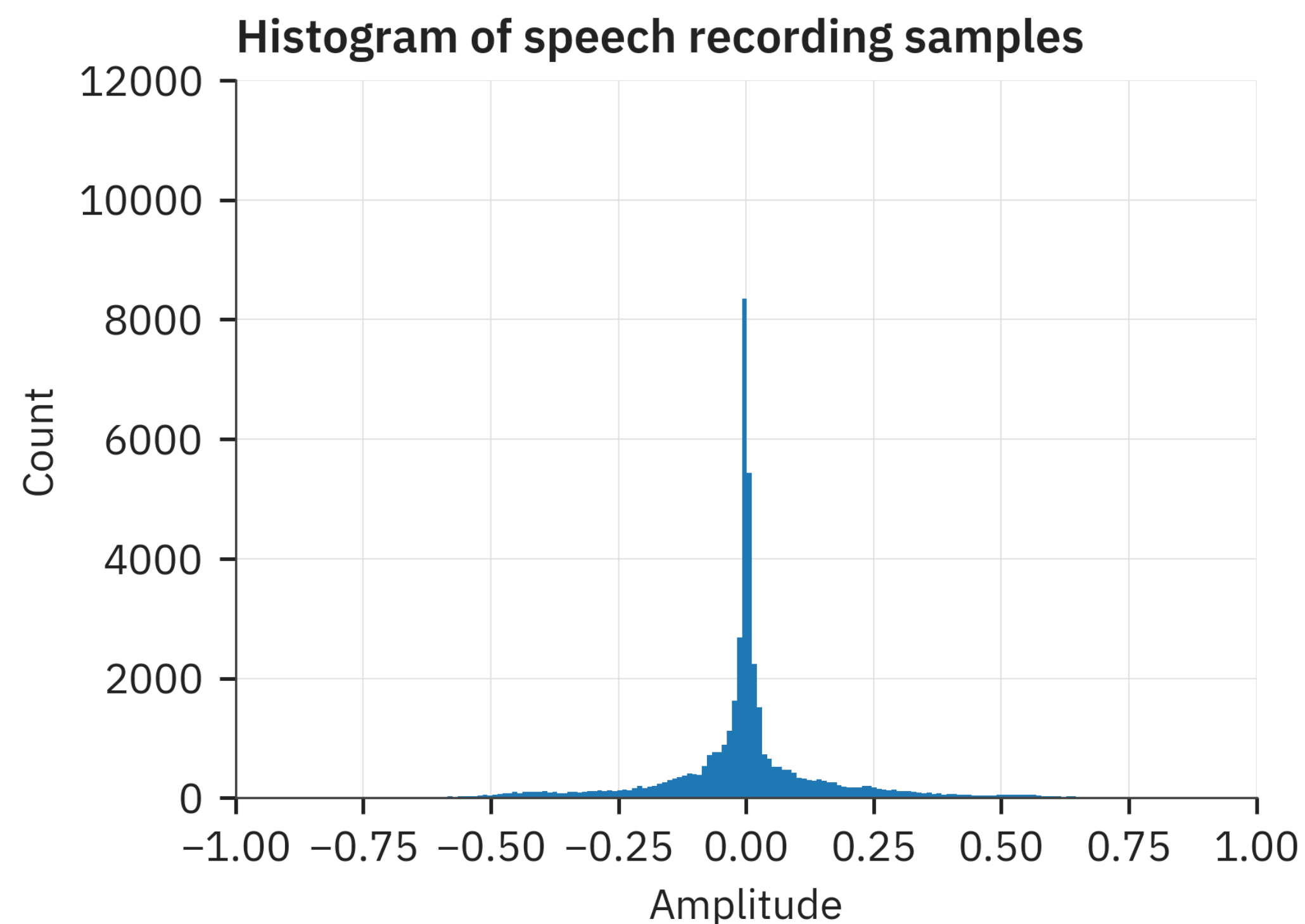


Coding and expanding

- Quantize the squashed waveform
 - Better distribution of bits over amplitudes
 - Undo the sigmoid to decode the sound
 - Result: Less bits needed for more dynamic range
- Many variants
 - μ -law / mu-law / u-law (US & Japan)
 - A-law (Europe)
 - Most often 64kbps audio (8-bit samples at 8kHz)
 - 8-bit u-law is equivalent to 12-bit normal encoding in sound quality

Differential PCM

- We can instead encode the differences between successive samples
 - Why is that better? Even heavier tails! → even fewer bits!



Differential PCM

July 29, 1952

C. C. CUTLER

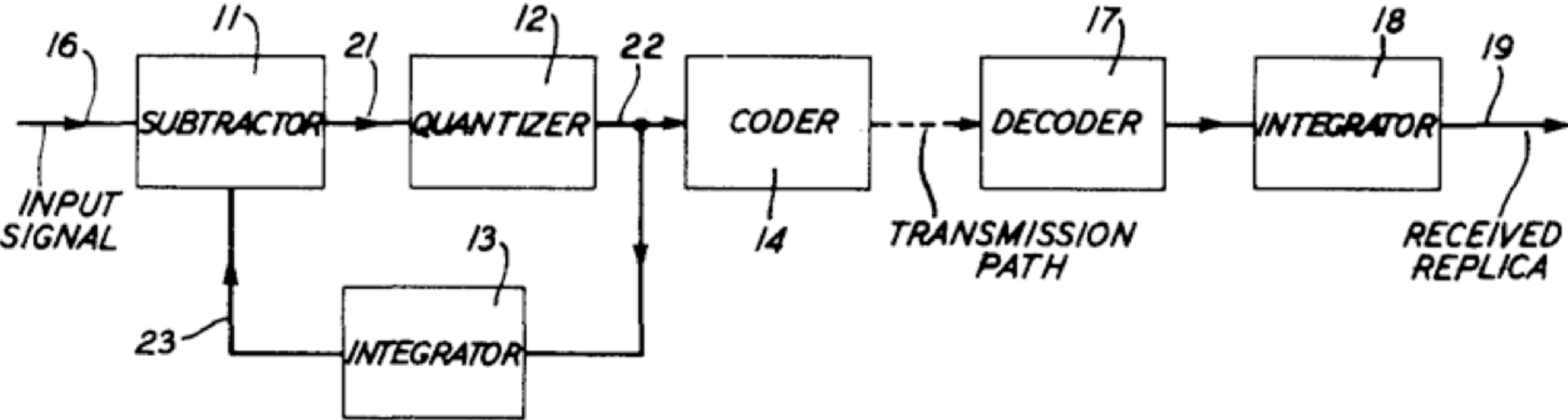
2,605,361

DIFFERENTIAL QUANTIZATION OF COMMUNICATION SIGNALS

Filed June 29, 1950

3 Sheets-Sheet 1

FIG. 1

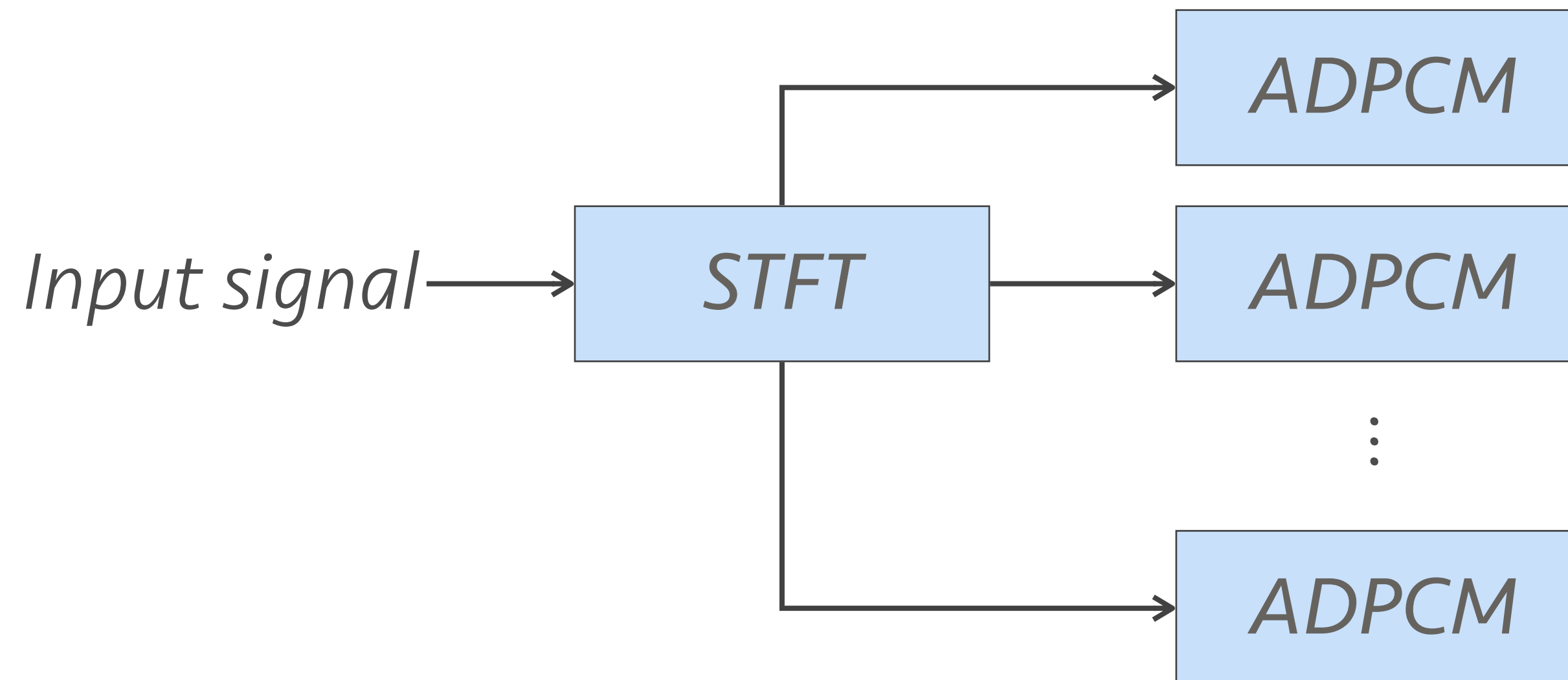


Adaptive PCM

- Why just take the difference?
 - How about we make a prediction of what is coming up and encode that error?
 - Even heavier tails, but more processing
- Prediction: $x(t) = \sum_i a_i x(t - i)$
 - Find optimal a 's for a given signal

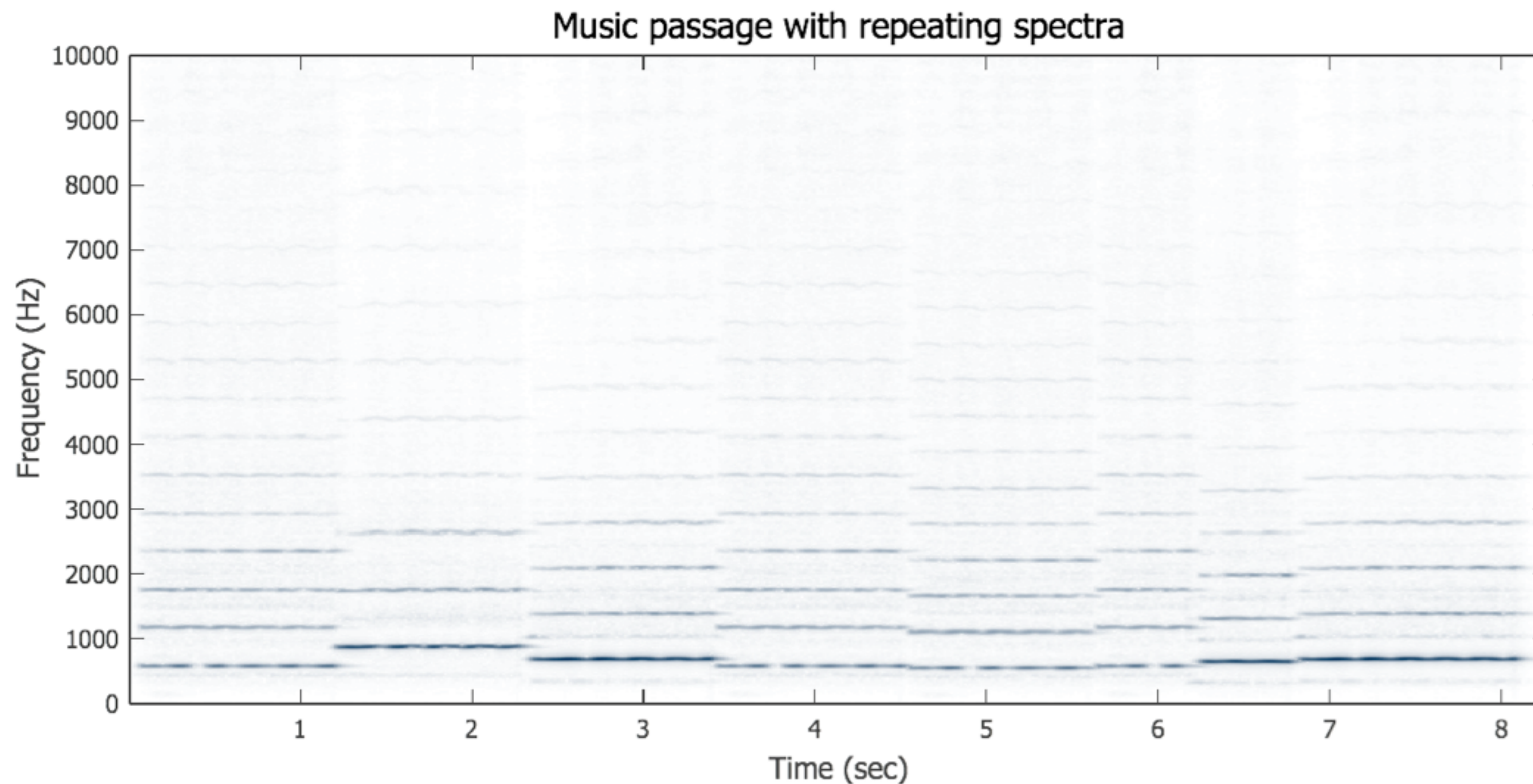
Split-band ADPCM

- And we can do that over multiple sub-bands
 - Break down the signal into multiple frequency bands and decimate (e.g. with an STFT)
 - Perform ADPCM on each band



Vector quantization

- Quantize spectra so that we obtain a small dictionary of repeating patterns
 - Example case

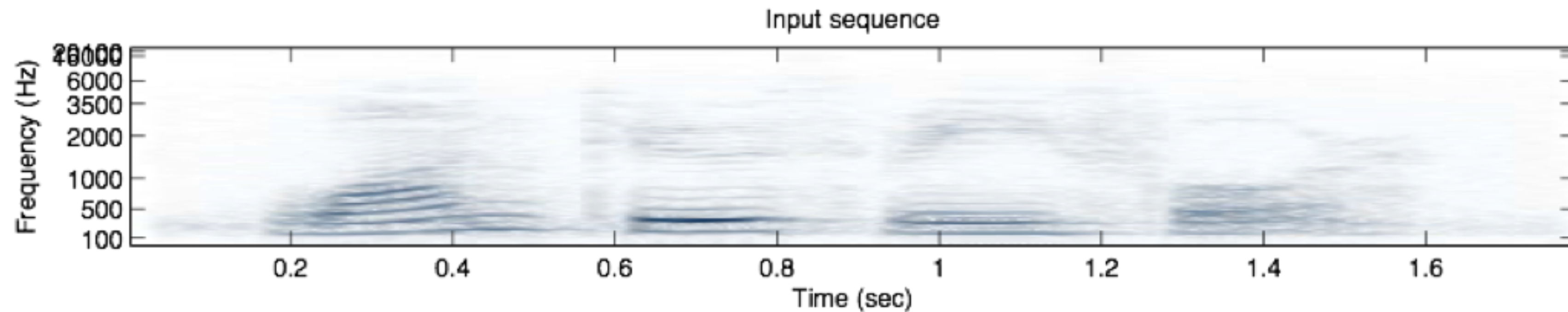


Key idea

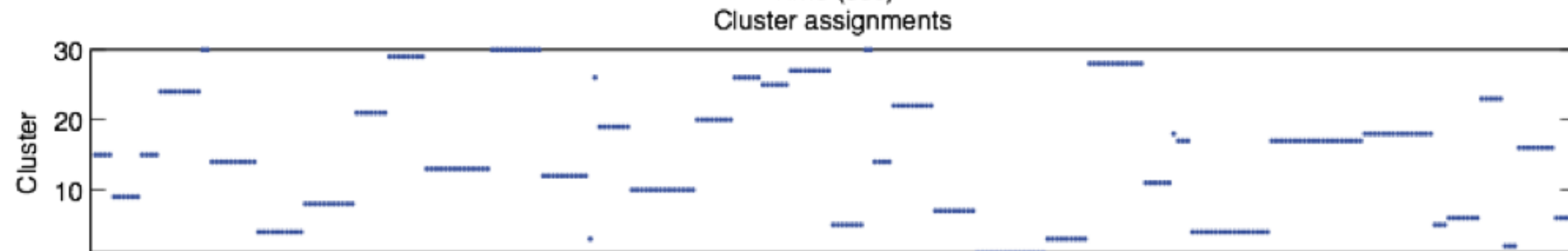
- Find representative spectra and make a dictionary
 - We can use various quantization schemes for that
- Encode sounds by matching spectra to dictionary
 - We only need to keep dictionary indices
 - No need to send over entire signal spectrum

VQ example

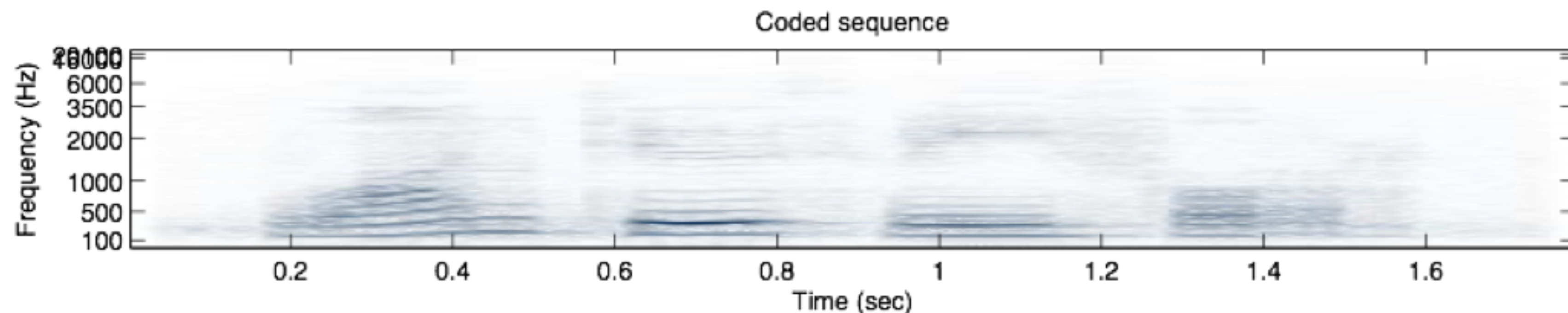
- Speech coding



Raw data:
1025 x 19 complex numbers
= 5,215,200 bytes



Dictionary data:
30 x 1025 amplitudes
= 246,000 bytes



Sent data:
model + 319 indices
= 246,319 bytes

VQ and dictionary coding

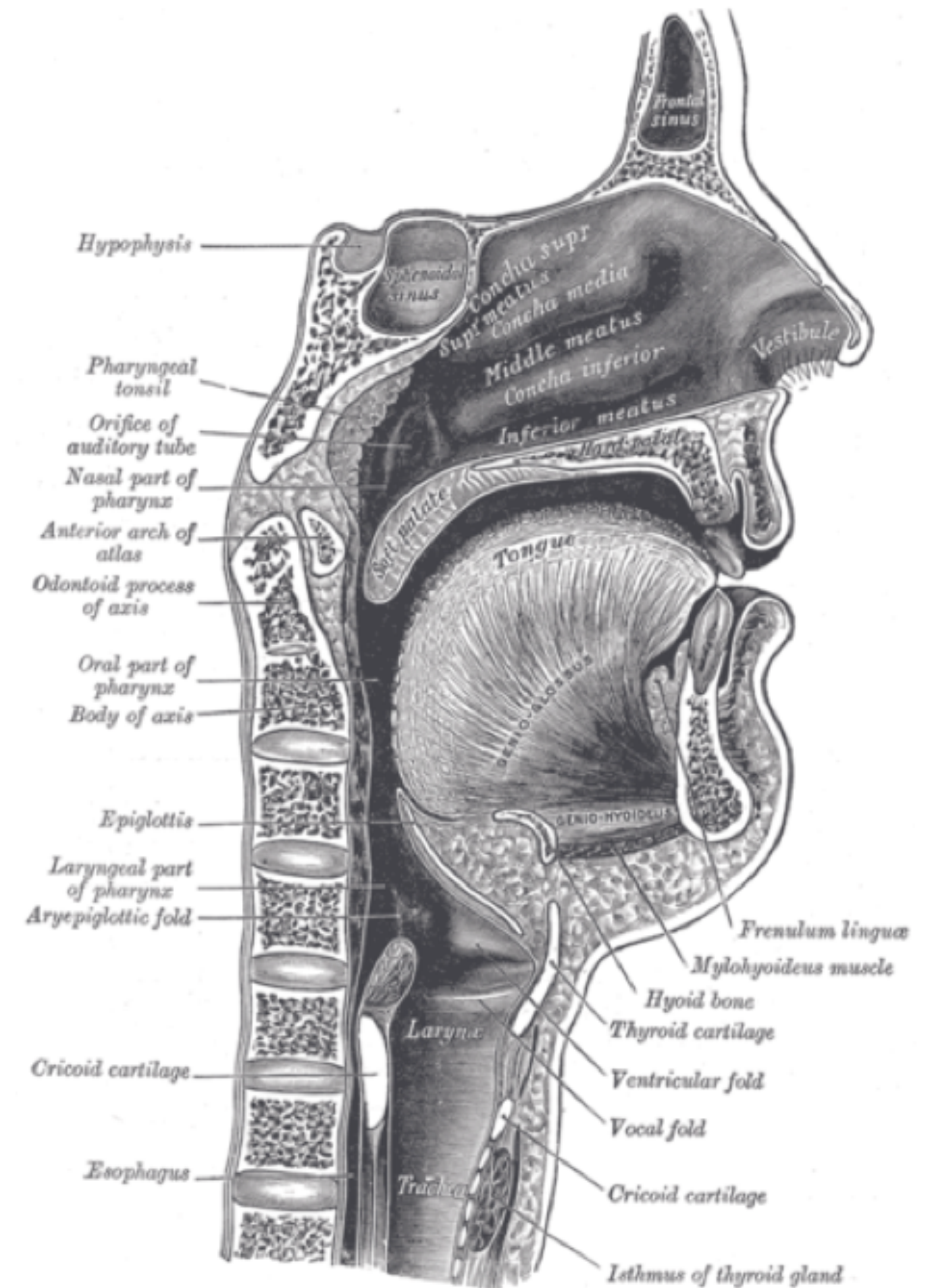
- Can be applied in various ways
- Dictionary extraction
 - Quantization, k-means, sparse coding, ...
- Applied on
 - Spectra, multi-channel audio, frames, ...

Voice compression

- We can also use a source-filter model
- Very useful for speech
 - Source is a glottal pulse-like signal
 - Filter mimics our vocal cavities
 - Just like speech synthesis
- How do we set this up?

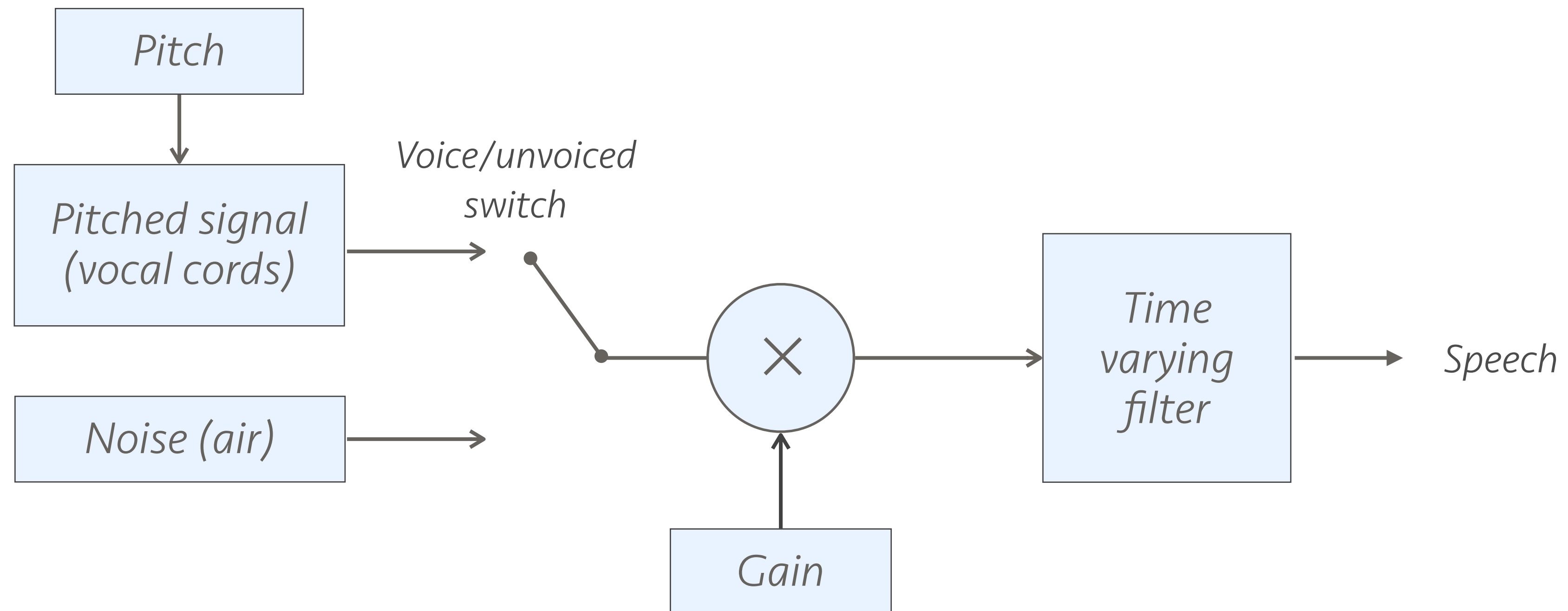
Modeling voice

- Multiple components
 - An excitation (vocal cords)
 - A filter (vocal cavities)
 - Noise (breath/air sounds)
 - Pitch, gain etc.
- We can encode the signal-producing parameters instead of the actual voice signal



Linear Prediction Coding

- Putting it all together:



- We just have to estimate all of the above parameters from a speech input to encode it in this compressed form

Estimating pitch

- We already covered that earlier in class
 - E.g. using autocorrelation
- Track pitch over a window of time
 - If pitch is unstable or non-existent
 - Flip voiced/unvoiced switch
 - Otherwise use estimate to set pitch value

The filter

- We model the sound using:

Input to encode $\longrightarrow$

$$s(t) = \sum_{i=1}^N a_i s(t-i) + G \cdot n(t)$$

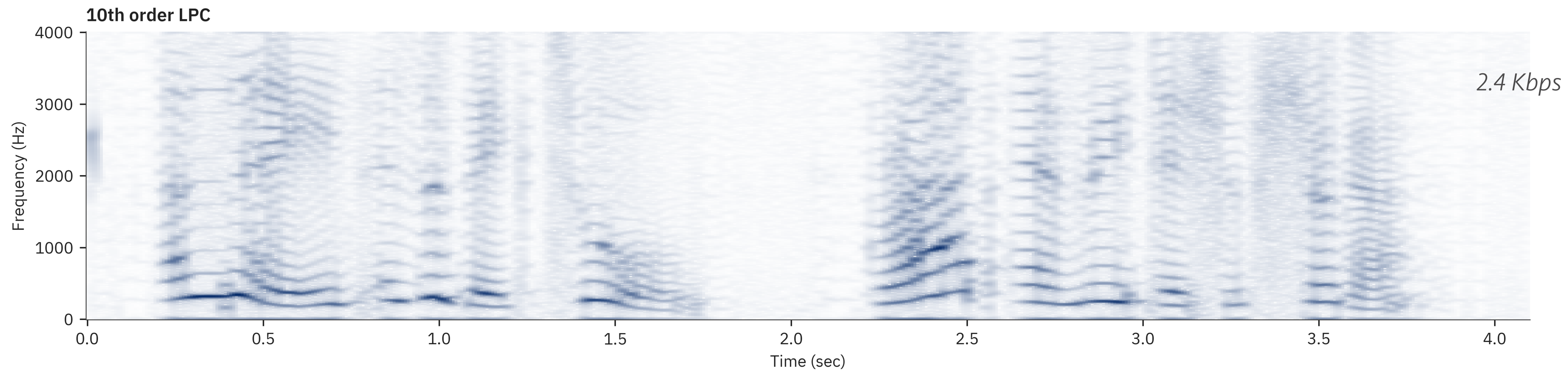
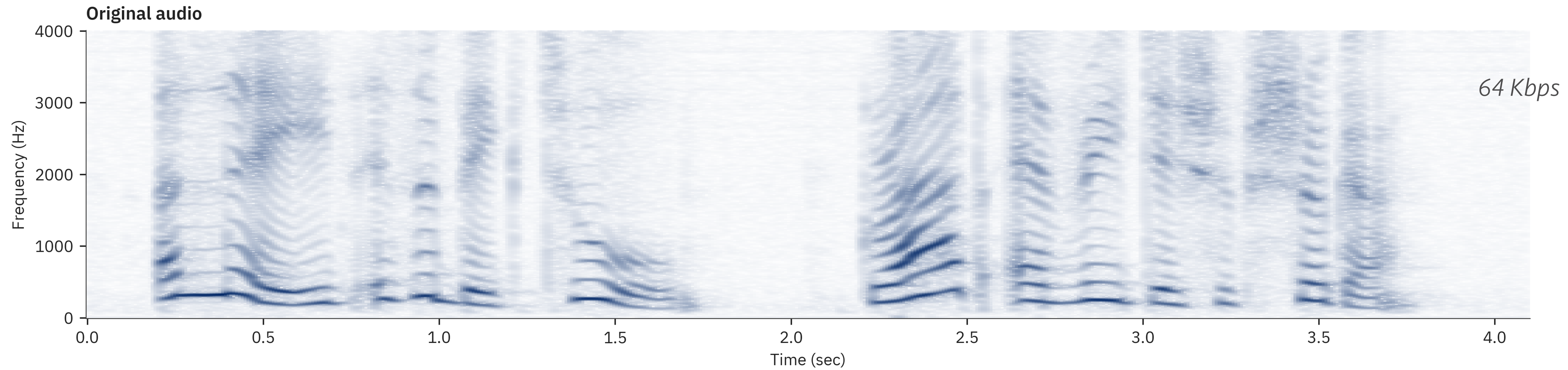
Filter (points to a_i) *Noise gain* (points to G) *Noise* (points to $n(t)$)

- Looks familiar?

- We thus need to find the a 's that minimize:

$$e(t) = s(t) - \sum_{i=1}^N a_i s(t-i)$$

Example coding



Details and elaborations

- Usual parameters
 - 10th order model, 20-30ms window
 - Results in 300-500 parameters per second
- Variations
 - Various kinds of excitation waveforms
 - Deltas, voice pulse, dictionary-based, ...
 - Varying representations of filter
 - Coefficients are not robust to errors

What about multi-source audio?

- LPC models are not appropriate for music and general audio signals
 - Each source has its own pitch and settings
- We need to use different methodologies
 - Lossless techniques
 - Perceptual Audio Coding (PAC)

Lossless coding

- Lossless coding ensures that we can reconstruct the exact same waveform
 - No data loss whatsoever
- Many different algorithms
 - AAC, FLAC, shorten, ...
- Can give us roughly 2× compression
 - But not much more

Generic audio lossless coders

- Two steps
 - Model the predictable part with a filter
 - Model what's left with traditional data coding
- Works well for time series
 - First step removes temporal correlations
 - Allows second step to use standard data compression

Example coding: the “shorten” algorithm

- First step:
 - Model the signal using predictive coding:

$$\hat{s}(t) = \sum_{i=1}^N a_i s(t - i)$$

- Re-estimate every 256 samples
- Calculate the prediction error (a.k.a. *the residual*)

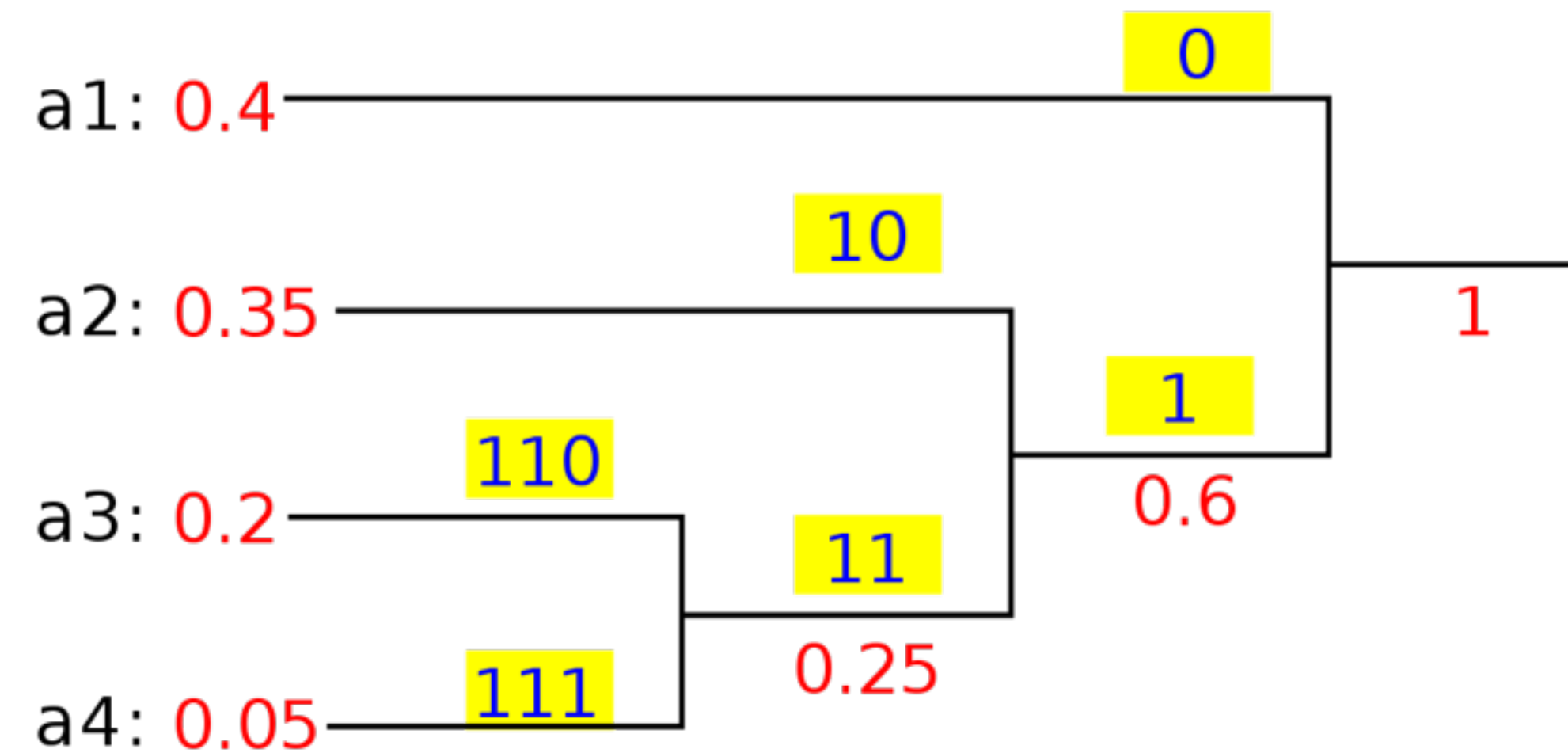
$$e(t) = s(t) - \hat{s}(t)$$

Residual coding

- Residual coding is “white”
 - It has minimized temporal correlations, i.e., the samples are maximally statistically independent
 - that means that we don’t have to worry about time
- Standard model to use then is Huffman coding
 - Popular way to losslessly compress raw data

Huffman coding example

- Compress the four symbols $\{a_1, a_2, a_3, a_4\}$
 - Given that $P(a_1)=0.4$, $P(a_2)=0.35$, $P(a_3)=0.2$, $P(a_4)=0.05$



- Final code:
 - $a_1=0$, $a_2=10$, $a_3=110$, $a_4=111 \rightarrow 1.85\text{b/symbol}$

But we need more ...

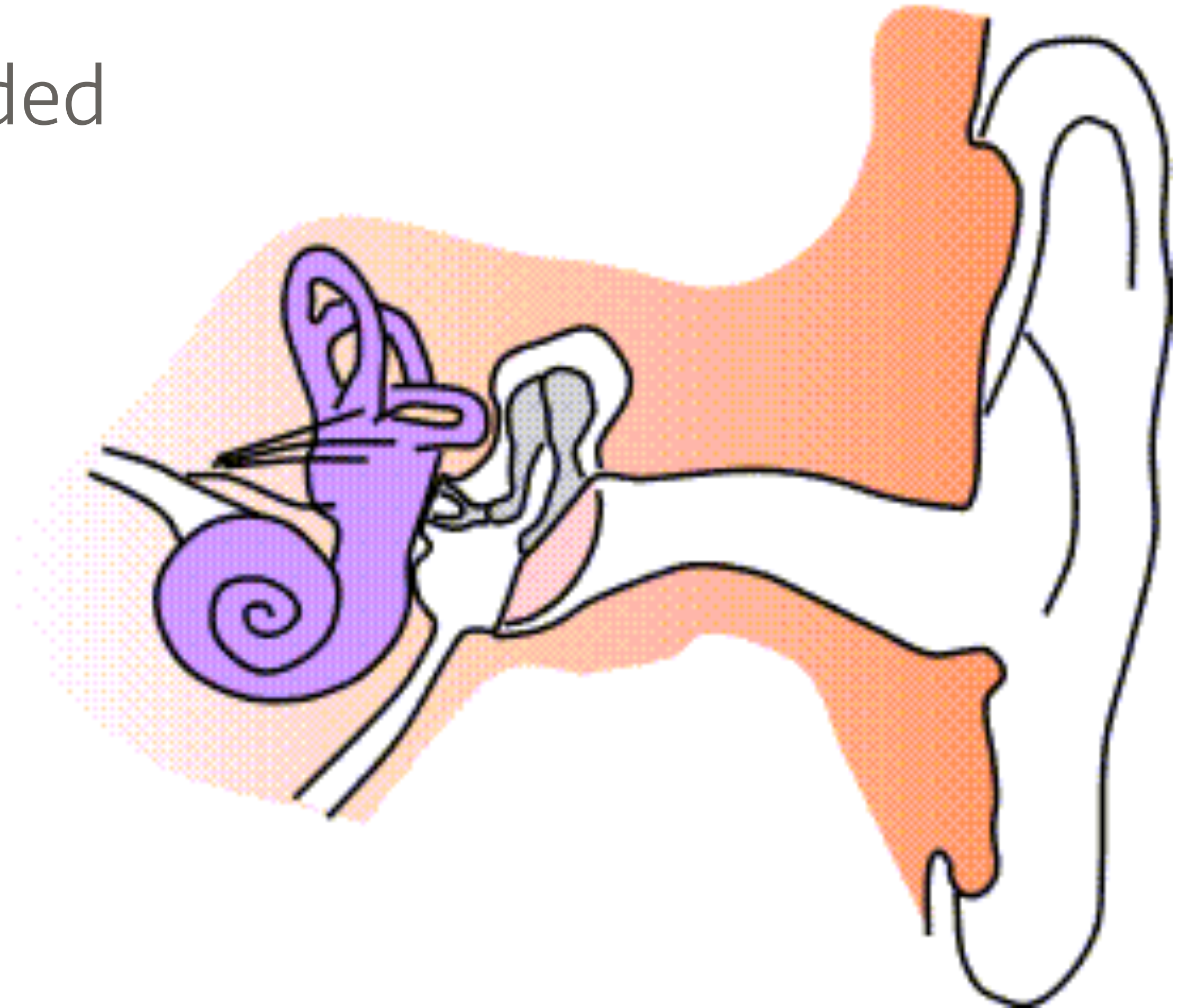
- Even the best encoders are not good enough
 - At best we get 2-3× compression
- With audio we can afford to lose some information
 - What should the tradeoffs be?
 - Any ideas on what to remove?

Perceptual Audio Coding

- Take advantage of the hearing irregularities
 - Use them to avoid encoding all the audio data
 - Don't encode things we can't hear
- Biggest gain comes from *masking effects*
 - Primarily frequency masking

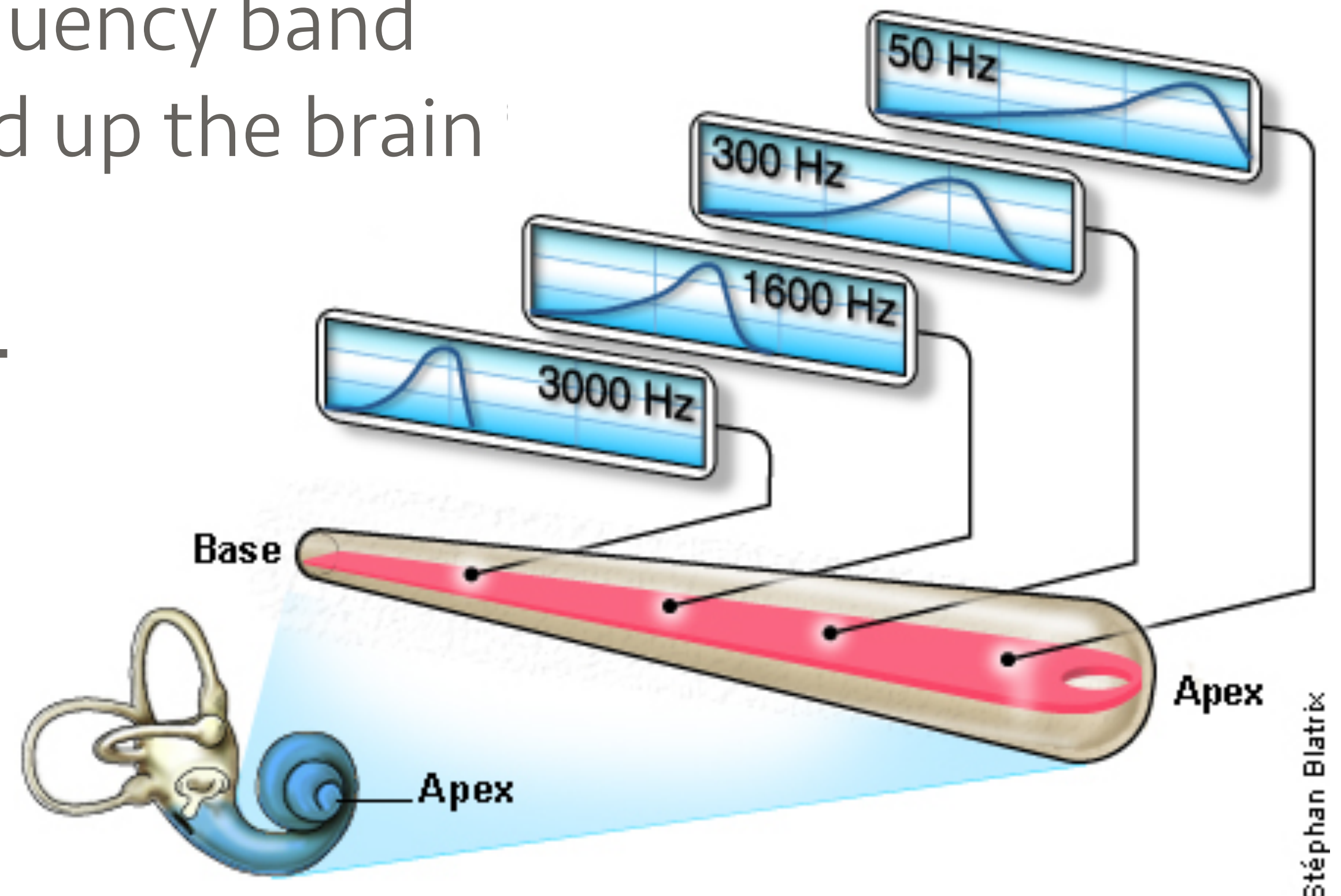
Back to hearing

- Remember the cochlea?
 - Vibrations get frequency-coded
- It turns out to be lossy
 - We will use that!



The cochlea

- Converts mechanical vibration to electrical
 - But first it breaks down sound by frequency
 - A membrane across the cochlea resonates at different parts according to energy at each frequency band
 - Hair cells pick up signal and send up the brain
- Roughly computes an STFT
 - Very roughly!



Masking and critical bands

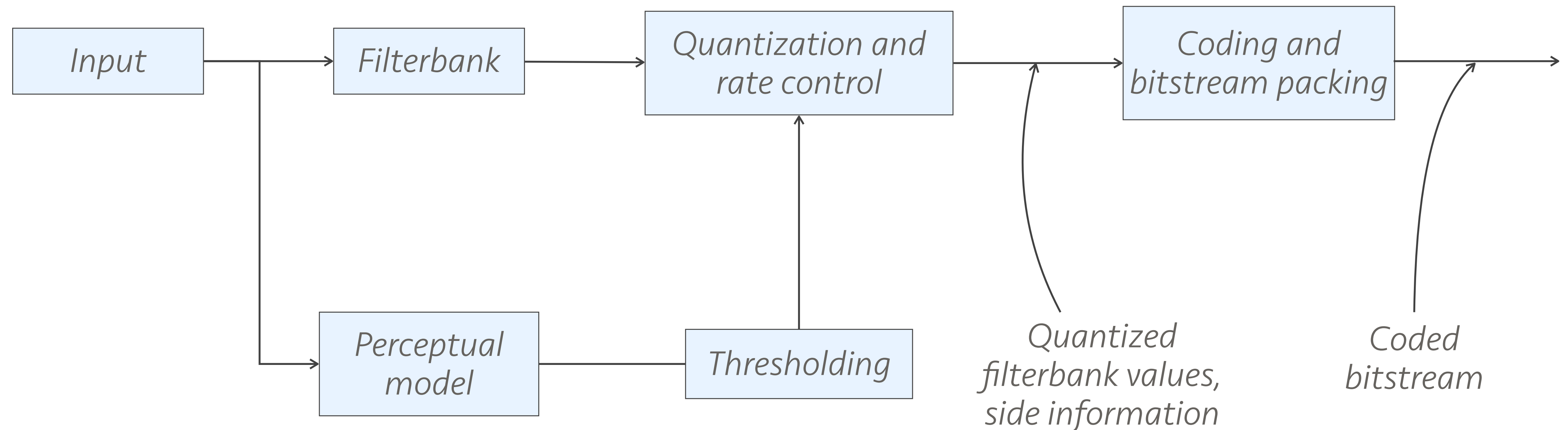
- The basilar membrane is split into frequency bands
 - a.k.a. the critical bands
- When different frequencies excite the same band, mostly the dominant one gets encoded
- Example demo
 - Two tones close in frequency
 - 2nd tone with decreasing amplitude gets masked after a while
 - Two tones distant in frequency
 - There is no masking with decreasing amplitude

What this means

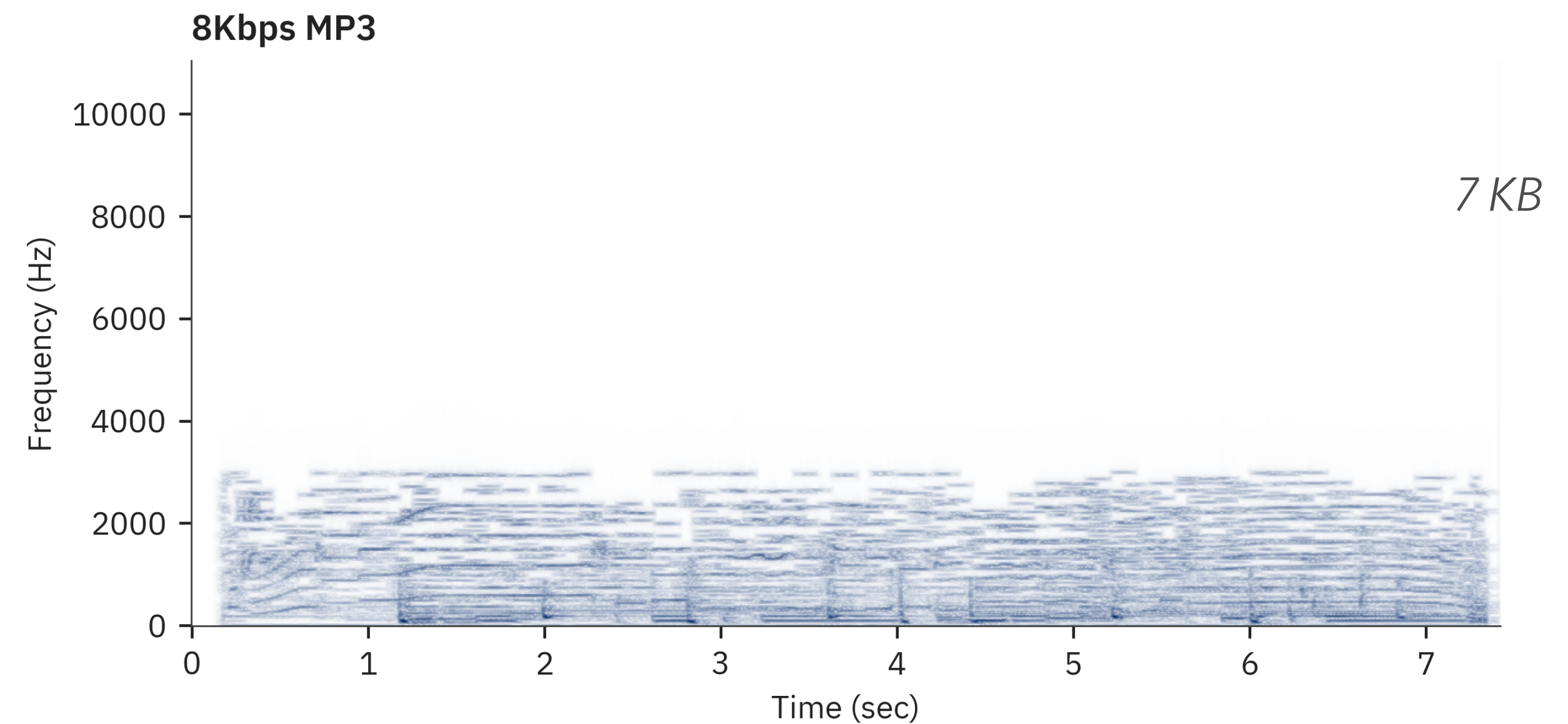
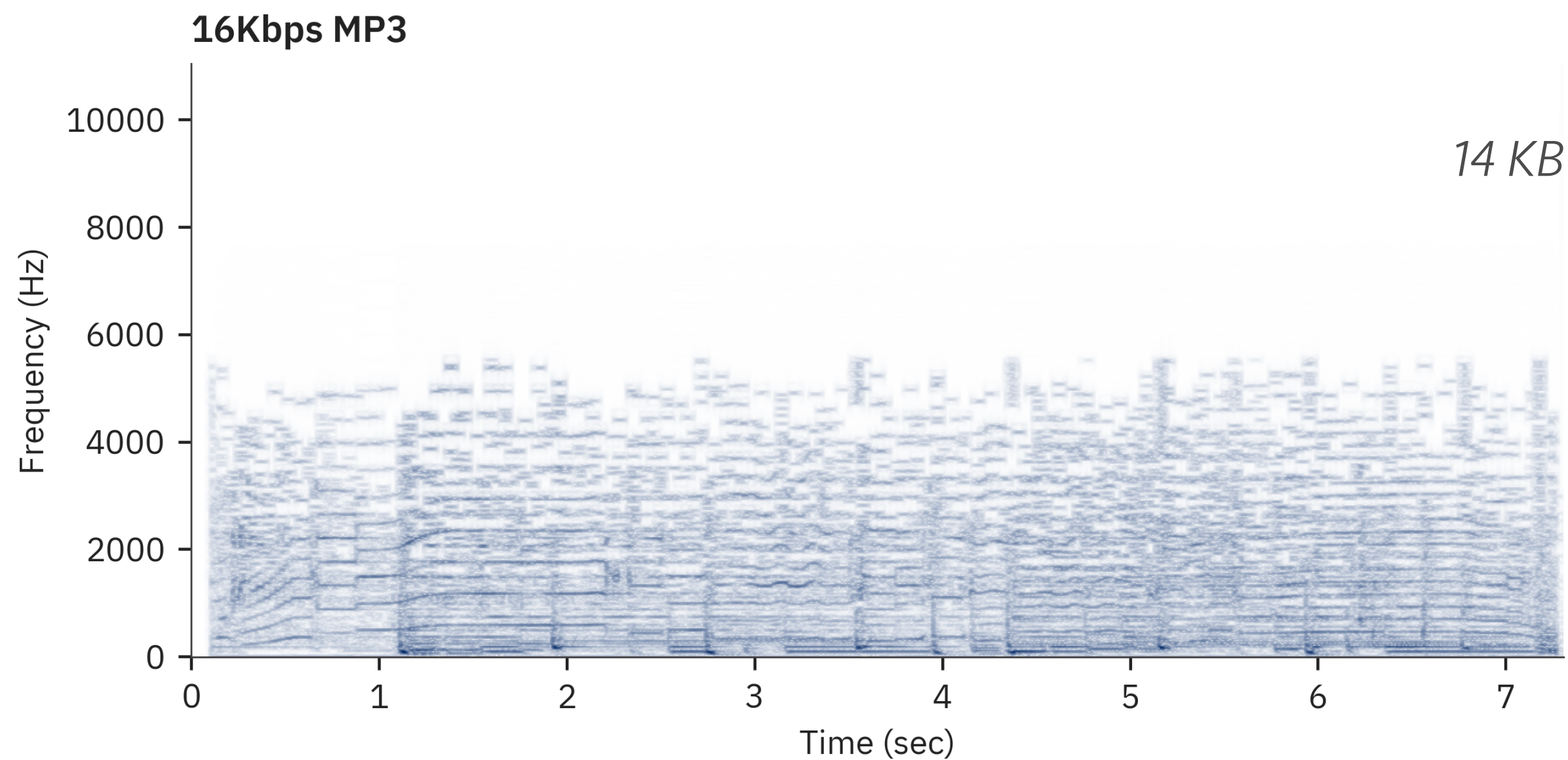
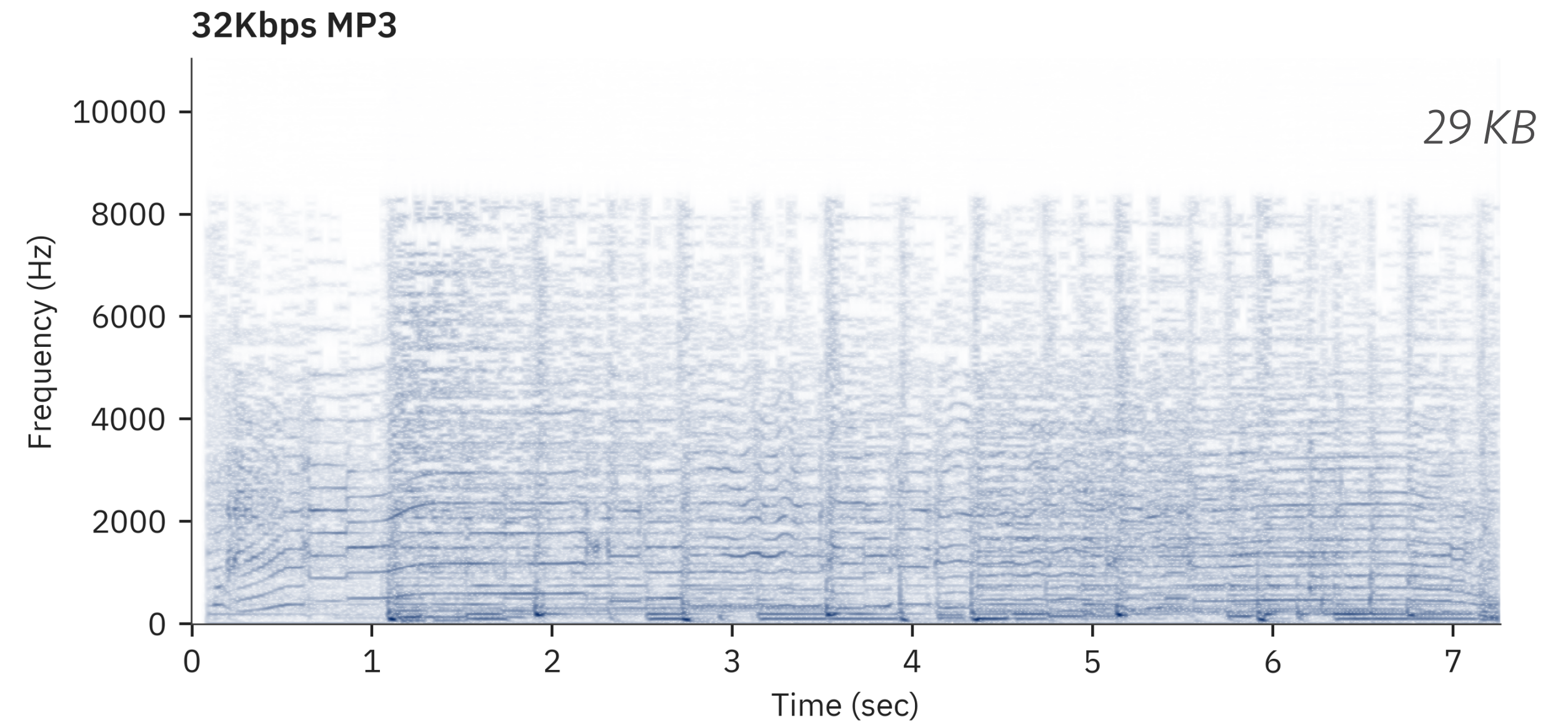
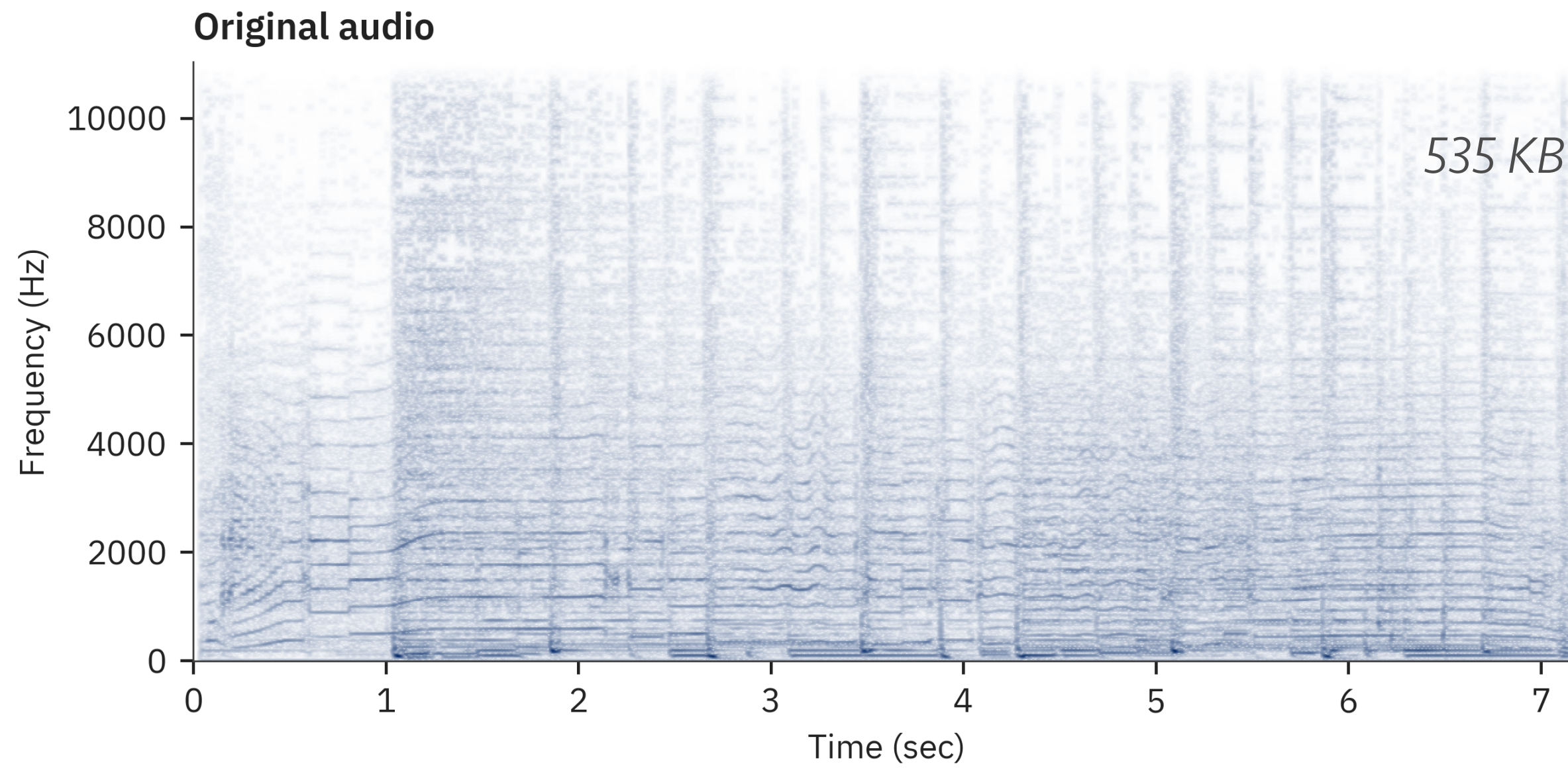
- When coding a spectrum we don't need to keep all the information
 - We can throw away the masked frequencies
 - Which can save us space!
- Perceptual model
 - Dictates what to ignore
 - E.g. frequency temporal, stereo, etc. masking

Perceptual Audio Coder (PAC) pipeline

- Use a high-resolution filterbank
 - Decide what to keep and what to throw away based on a perceptual model



MP3 Examples



And there's more ...

- Multi-channel encoding
 - Stereo, 5.1, 7.1, etc.
- Hybrid models
 - Switch from LPC to PAC according to input
- Fancy neural vocoders and encoders

The landscape

- Lossless encoders
 - Shorten, Free Lossless Audio Codec (FLAC), Apple Lossless (ALAC), MPEG-4 ALS, Windows Media Audio 9 Lossless (WMA lossless), Monkey's audio, ...
- Speech encoders
 - G.711, G.718, G.722, G.729, ...
- Lossy coders
 - MP3, MPEG-4, WMA, AAC, ...
- Most recent trend: Neural coders

Recap

- Simple waveform coding
 - Compressing, etc.
- Linear predictive models
 - Good for speech and source/filter models
- Lossless coding
- Perceptual Audio Coding
 - Good for busy wideband content (e.g. music)

Reading material

- Speech coding overview
 - <https://ieeexplore.ieee.org/document/326413>
- Linear Predictive Coding
 - <http://ieeexplore.ieee.org/xpl/articleDetails.jsp?tp=&arnumber=1890>
- MPEG audio compression
 - <http://www.ee.columbia.edu/~dpwe/e6820/papers/Pan95-mpega.pdf>

Logistics

- We are done with the labs!
 - If you are registered for 4 credit hours you have a project to do!
 - If you are registered for 3 credit hours, you are welcome to too
 - You'll get extra credit to make up for any lost lab points
- Final project guidelines
 - No team projects (unless they are cleanly split in parts)
 - Due on last day or finals (no exceptions)
 - Project proposal due next week
 - Simple 1-2 pager with deliverables